Bashir Agro Admin Panel
Phase 4: Task Template Management Module

Phase Scope

Implement ONLY Task Template Management after Phase 3.

Included:
- Task template database
- CRUD
- Search
- Filters
- Pagination
- Validation
- Activity logs
- Tests

Not included:
- Task assignment
- Farmer allocation
- Flutter app
- REST API
- Task submission
- Photo upload from farmer
- GPS validation
- PostGIS

Objective

Convert the existing Task Management section into a fully dynamic Laravel + PostgreSQL module while preserving the existing Bengali agricultural Admin Panel design.

Database Table

Create:

task_templates

Fields:

id
task_code
name
category
description
instructions
required_photo_count
require_realtime_photo
allow_gallery_upload
require_geofence
recurrence_type
status

created_by
created_at
updated_at
deleted_at

Field Explanation

task_code:
Generated automatically:
TASK-000001

name:
Task title.

category:
Crop Maintenance, Irrigation, Harvesting, etc.

description:
Short explanation.

instructions:
Detailed task instructions.

required_photo_count:
Number of required photos.

require_realtime_photo:
Whether camera capture is required.

allow_gallery_upload:
Whether old gallery images are allowed.

require_geofence:
Future GPS validation requirement.

recurrence_type:
once, daily, weekly, monthly.

status:
active/inactive.

Model

Create:

app/Models/TaskTemplate.php

Requirements:

- Use SoftDeletes
- Add fillable fields
- Add creator relationship

Relationship:

TaskTemplate belongsTo User through created_by.

Task Code Generation

Create:

generateNextTaskCode()

inside TaskTemplate model.

Example:

TASK-000001
TASK-000002
TASK-000003

The backend generates the code.
Admin must not enter it manually.

Controller

Create:

app/Http/Controllers/Admin/TaskTemplateController.php

Methods:

index()
create()
store()
show()
edit()
update()
destroy()

Use Eloquent.
Keep controllers clean.

Form Requests

Create:

StoreTaskTemplateRequest.php

UpdateTaskTemplateRequest.php

Validate:

name
category
description
instructions

required_photo_count
boolean options
recurrence_type
status

Create Workflow

Admin creates task template.

Flow:

Blade Form
↓
Laravel Validation
↓
Generate Task Code
↓
Save PostgreSQL
↓
Create Activity Log
↓
Redirect

Use DB::transaction().

Task List Page

Create:

resources/views/tasks/index.blade.php

Display:

Task Code
Task Name
Category
Required Photos
Camera Requirement
Gallery Permission
Geofence Requirement
Recurrence
Status
Actions

Remove hard-coded task data from JavaScript.

Search

Implement server-side search.

URL example:

/admin/tasks?search=fertilizer

Search:

task_code
name
category

Filters

Create:

■■
■■■■■■■
■■■■■■■■■■

Example:

/admin/tasks?status=active

Search and filters must work together.

Pagination

Use Laravel pagination.

Recommended:

20 tasks per page.

Keep query parameters using:

withQueryString().

Create and Edit UI

Convert existing HTML design into Blade.

Preserve:

- Bengali labels
- Green theme
- Cards
- Switch design
- Forms
- Responsive layout

Fields:

Task Name
Category
Description
Instructions
Photo Count

Realtime Camera
Gallery Upload
Geofence
Recurrence
Status

Soft Delete

Use Laravel SoftDeletes.

Do not permanently delete task templates because future assignments may depend on historical task information.

Activity Logging

Create logs:

task.created
task.updated
task.deleted
task.activated
task.deactivated

Store:

user_id
action
subject_type
subject_id
description
old_values
new_values
ip_address

Routes

Add resource routes:

```
Route::resource(
'tasks',
TaskTemplateController::class
);
```

Under:

/admin/tasks

Testing

Create:

tests/Feature/TaskTemplateManagementTest.php

Test:

- Authentication
- Create task
- Task code generation
- Validation
- Update task
- Soft delete
- Search
- Filters
- Pagination

Security

Use:

- Authentication middleware
- CSRF protection
- Form Requests
- Authorization
- Mass assignment protection
- Server-side validation

Never trust JavaScript validation alone.

Antigravity Prompt

```
Proceed to PHASE 4 ONLY: Task Template Management Module.

Use the existing Bashir Agro Laravel project after Phase 3.

Do not redesign the UI.

Do not implement:
- Task Assignment
- Farmer allocation
- Flutter app
- REST API
- Task submission
- Photo upload
- GPS validation
- PostGIS

Implement:

- task_templates table
- TaskTemplate model
- TaskTemplateController CRUD
- StoreTaskTemplateRequest
- UpdateTaskTemplateRequest
- Task code generation
- Search
- Filters
- Pagination
- Soft delete
```

```
- Activity logging
- Feature tests

Preserve Bengali UI, green theme, cards, tables and responsive design.

Remove hard-coded task data from JavaScript.

Database must be the source of truth.

Run migrations.
Run php artisan test.
Run vendor/bin/pint.

Report:
- files created
- files modified
- database changes
- routes added
- test results

STOP after Phase 4.
Do not start Phase 5 automatically.
```